TABLE OF CONTENTS

SQL

SQL for Data Analytics

The Complete Reference Guide

SELECT · JOIN · Aggregation · Window Functions · CTEs · Subqueries
Date Functions · String Functions · Analytics Patterns · Performance

CHAPTERS COVERED

1. SQL Fundamentals & SELECT Statement

Retrieving data, column selection, aliases, DISTINCT, data types

SQL (Structured Query Language) is the standard language for querying relational databases. Every analytics workflow begins with SELECT — the command that retrieves data from one or more tables. Understanding its full syntax is the foundation of all advanced SQL work.

1.1 Basic SELECT Syntax

```
SELECT column1, column2, column3
FROM   table_name;

-- Select all columns (use sparingly in production)
SELECT *
FROM   employees;

-- Select with table alias
SELECT e.employee_id, e.first_name, e.salary
FROM   employees AS e;
```

Basic SELECT forms

1.2 Column Aliases

```
SELECT
    first_name                      AS "First Name",
    last_name                       AS "Last Name",
    salary * 12                     AS annual_salary,
    department_id                   AS dept,
    ROUND(salary / 1000.0, 2)       AS salary_k
FROM employees;
```

Column aliases using AS keyword

1.3 DISTINCT – Remove Duplicates

```
-- Unique departments
SELECT DISTINCT department_id
FROM   employees;

-- Unique combination of columns
SELECT DISTINCT department_id, job_id
FROM   employees;

-- Count of unique values
SELECT COUNT(DISTINCT department_id) AS unique_departments
FROM   employees;
```

1.4 SQL Data Types Reference

Data Type	Description	Example Use
INT / INTEGER	Whole numbers	employee_id, age, quantity

BIGINT	Large whole numbers	row_count, population
DECIMAL(p,s)	Exact fixed-point	salary DECIMAL(10,2)
FLOAT / REAL	Approximate numeric	scientific measurements
VARCHAR(n)	Variable-length text	name VARCHAR(100)
CHAR(n)	Fixed-length text	country_code CHAR(3)
TEXT	Unlimited text	description, notes
DATE	Date only (YYYY-MM-DD)	hire_date, birth_date
DATETIME / TIMESTAMP	Date + time	order_timestamp
BOOLEAN / BIT	True/False (1/0)	is_active, is_deleted
NULL	Absence of value	optional fields

Tip: Always choose the smallest data type that fits your data. Use DECIMAL for money, not FLOAT (floating-point rounding causes cents errors).

1.5 SQL Execution Order

SQL clauses execute in this order regardless of how you write them:

Step	Clause	What Happens
1	FROM / JOIN	Tables are loaded and joined
2	WHERE	Rows are filtered before grouping
3	GROUP BY	Rows are grouped for aggregation
4	HAVING	Groups are filtered after aggregation
5	SELECT	Columns are selected / expressions evaluated
6	DISTINCT	Duplicate rows removed
7	ORDER BY	Result set is sorted
8	LIMIT / TOP	Row count is restricted

2. Filtering, Sorting & Limiting

WHERE clause, logical operators, pattern matching, ORDER BY, LIMIT/TOP

2.1 WHERE Clause – Comparison Operators

```
SELECT * FROM orders
WHERE  order_date >= '2024-01-01'          -- greater than or equal
  AND  status     =  'completed'            -- equality
  AND  amount     <> 0                      -- not equal (also != )
  AND  discount   IS NOT NULL;              -- NULL check

-- Operators: =  !=  <>  <  >  <=  >=  IS NULL  IS NOT NULL
```

2.2 AND / OR / NOT Logic

```
-- Parentheses clarify precedence (AND binds tighter than OR)
SELECT * FROM employees
WHERE  (department_id = 10 OR department_id = 20)
  AND   salary > 50000
  AND   NOT job_id = 'INTERN';

-- Equivalent using IN
SELECT * FROM employees
WHERE  department_id IN (10, 20)
  AND   salary > 50000;
```

2.3 BETWEEN, IN, NOT IN

```
-- BETWEEN (inclusive on both ends)
SELECT * FROM orders
WHERE  order_date BETWEEN '2024-01-01' AND '2024-12-31'
  AND  amount     BETWEEN 100 AND 5000;

-- IN list
SELECT * FROM products
WHERE  category IN ('Electronics', 'Appliances', 'Gadgets');

-- NOT IN  (careful: fails silently if list contains NULL)
SELECT * FROM employees
WHERE  department_id NOT IN (SELECT department_id FROM inactive_depts
                             WHERE department_id IS NOT NULL);
```

2.4 LIKE & Pattern Matching

```
-- % matches zero or more characters
SELECT * FROM customers WHERE email LIKE '%@gmail.com';
SELECT * FROM products  WHERE name  LIKE 'Apple%';     -- starts with
SELECT * FROM products  WHERE name  LIKE '%Pro%';      -- contains

-- _ matches exactly one character
SELECT * FROM codes WHERE code LIKE 'A_C';             -- A?C (3 chars)

-- ILIKE (PostgreSQL) – case-insensitive
SELECT * FROM customers WHERE name ILIKE 'john%';
```

```
-- REGEXP / RLIKE (MySQL, BigQuery)
SELECT * FROM users WHERE phone REGEXP '^[0-9]{10}$';
```

2.5 ORDER BY – Sorting Results

```
-- Single column ascending (default)
SELECT * FROM employees ORDER BY salary;

-- Descending
SELECT * FROM employees ORDER BY salary DESC;

-- Multiple columns (salary DESC, then name ASC for ties)
SELECT employee_id, first_name, department_id, salary
FROM   employees
ORDER  BY department_id ASC, salary DESC;

-- Order by expression or alias
SELECT first_name, salary * 12 AS annual_salary
FROM   employees
ORDER  BY annual_salary DESC;

-- NULLS FIRST / NULLS LAST (PostgreSQL, Oracle)
SELECT * FROM employees ORDER BY manager_id NULLS LAST;
```

2.6 LIMIT / TOP / FETCH

```
-- MySQL, PostgreSQL, SQLite, BigQuery
SELECT * FROM orders ORDER BY order_date DESC LIMIT 10;

-- With offset (pagination: page 3, 10 rows per page)
SELECT * FROM orders ORDER BY order_id LIMIT 10 OFFSET 20;

-- SQL Server / Sybase
SELECT TOP 10 * FROM orders ORDER BY order_date DESC;

-- Standard SQL (SQL Server 2012+, Oracle 12c+)
SELECT * FROM orders
ORDER  BY order_date DESC
FETCH  FIRST 10 ROWS ONLY;

-- Top 10% (SQL Server)
SELECT TOP 10 PERCENT * FROM orders ORDER BY amount DESC;
```

Tip: Always use ORDER BY with LIMIT/TOP. Without ORDER BY, the "top" rows are arbitrary.

3. Aggregation & GROUP BY

Aggregate functions, GROUP BY, HAVING, ROLLUP, CUBE, GROUPING SETS

3.1 Core Aggregate Functions

Function	Description	Example Output
COUNT(*)	Counts all rows including NULLs	COUNT(*) → 1000
COUNT(col)	Counts non-NULL values	COUNT(email) → 892
COUNT(DISTINCT col)	Counts unique non-NULL values	COUNT(DISTINCT city) → 45
SUM(col)	Total of numeric column	SUM(revenue) → 4250000
AVG(col)	Mean (ignores NULLs)	AVG(salary) → 67500
MIN(col)	Smallest value	MIN(order_date) → 2020-01-01
MAX(col)	Largest value	MAX(salary) → 250000
STDEV(col)	Standard deviation	STDEV(score) → 12.4
VARIANCE(col)	Variance of values	VARIANCE(score) → 153.8
STRING_AGG(col,sep)	Concatenate values (SQL Server, PG)	A, B, C, D
GROUP_CONCAT(col)	Concatenate values (MySQL)	A,B,C,D
ARRAY_AGG(col)	Collect into array (PostgreSQL, BQ)	{1,2,3}

3.2 GROUP BY Examples

```
-- Sales by department
SELECT   department_id,
         COUNT(*)          AS employee_count,
         AVG(salary)       AS avg_salary,
         SUM(salary)       AS total_payroll,
         MAX(salary)       AS top_salary
FROM     employees
GROUP BY department_id
ORDER BY total_payroll DESC;

-- Group by multiple columns
SELECT   department_id, job_id,
         COUNT(*)   AS headcount,
         AVG(salary) AS avg_sal
FROM     employees
GROUP BY department_id, job_id
ORDER BY department_id, avg_sal DESC;
```

3.3 HAVING – Filter After Aggregation

```
-- Departments with more than 5 employees AND avg salary > 60,000
SELECT   department_id,
```

```
        COUNT(*)    AS emp_count,
        AVG(salary) AS avg_salary
FROM    employees
GROUP BY department_id
HAVING  COUNT(*) > 5
  AND   AVG(salary) > 60000
ORDER BY avg_salary DESC;

-- WHERE vs HAVING
-- WHERE  filters rows BEFORE grouping
-- HAVING filters groups AFTER aggregation
SELECT  category, SUM(revenue) AS total_rev
FROM    sales
WHERE   sale_date >= '2024-01-01'   -- filter rows first
GROUP BY category
HAVING  SUM(revenue) > 100000;      -- then filter groups
```

3.4 ROLLUP – Subtotals & Grand Totals

```
-- ROLLUP adds subtotal rows automatically
SELECT  region, category, SUM(sales) AS total_sales
FROM    sales_data
GROUP BY ROLLUP(region, category);
-- Produces: (region,category), (region,NULL=subtotal), (NULL,NULL=grand total)

-- SQL Server / MySQL syntax
SELECT  region, category, SUM(sales) AS total_sales
FROM    sales_data
GROUP BY region, category WITH ROLLUP;
```

3.5 CUBE & GROUPING SETS

```
-- CUBE: all possible combinations of subtotals
SELECT  year, quarter, region, SUM(revenue)
FROM    sales_data
GROUP BY CUBE(year, quarter, region);

-- GROUPING SETS: specify exactly which subtotals you want
SELECT  region, category, SUM(sales)
FROM    sales_data
GROUP BY GROUPING SETS (
    (region, category),   -- detail
    (region),             -- by region subtotal
    (category),           -- by category subtotal
    ()                    -- grand total
);

-- GROUPING() function: detect NULL from ROLLUP vs real NULL
SELECT  region,
        GROUPING(region) AS is_subtotal,
        SUM(sales)
FROM    sales_data
GROUP BY ROLLUP(region);
```

4. JOINs – All Types

INNER, LEFT, RIGHT, FULL OUTER, CROSS, SELF, multiple table joins

4.1 JOIN Types Overview

Join Type	Returns	Use When
INNER JOIN	Only matching rows from BOTH tables	Most common; excludes non-matches
LEFT JOIN (LEFT OUTER)	All rows from left + matches from right (NULLs for no match)	Keep all from primary table
RIGHT JOIN (RIGHT OUTER)	All rows from right + matches from left	Less common; swap to LEFT JOIN
FULL OUTER JOIN	All rows from BOTH tables (NULLs for gaps)	Find all unmatched rows
CROSS JOIN	Cartesian product of every row × every row	Generate combinations
SELF JOIN	Table joined to itself	Hierarchy, manager queries

4.2 INNER JOIN

```
-- Employees with their department names
SELECT e.employee_id, e.first_name, d.department_name, d.location_id
FROM   employees   AS e
INNER JOIN departments AS d ON e.department_id = d.department_id;

-- Multi-column join condition
SELECT *
FROM   orders o
INNER JOIN order_items oi
    ON o.order_id = oi.order_id AND o.store_id = oi.store_id;
```

4.3 LEFT JOIN

```
-- All employees, even those with no department assigned
SELECT e.employee_id, e.first_name, d.department_name
FROM   employees   AS e
LEFT JOIN departments AS d ON e.department_id = d.department_id;

-- Find employees WITHOUT a department (anti-join pattern)
SELECT e.employee_id, e.first_name
FROM   employees   AS e
LEFT JOIN departments AS d ON e.department_id = d.department_id
WHERE  d.department_id IS NULL;   -- <- key: filter on NULL from right side
```

4.4 FULL OUTER JOIN

```
-- All employees and all departments (matched or not)
SELECT e.first_name, d.department_name
FROM   employees   AS e
FULL OUTER JOIN departments AS d ON e.department_id = d.department_id;
```

```
-- MySQL doesn't support FULL OUTER JOIN — emulate with UNION:
SELECT e.first_name, d.department_name
FROM employees e LEFT  JOIN departments d ON e.department_id = d.department_id
UNION
SELECT e.first_name, d.department_name
FROM employees e RIGHT JOIN departments d ON e.department_id = d.department_id;
```

4.5 CROSS JOIN

```
-- All combinations of sizes and colours (3 × 4 = 12 rows)
SELECT s.size_name, c.colour_name
FROM   sizes  AS s
CROSS JOIN colours AS c;

-- Date spine: generate a row for every day of the year
SELECT d.date_val
FROM   date_dimension AS d
CROSS JOIN product_list AS p
WHERE  d.date_val BETWEEN '2024-01-01' AND '2024-12-31';
```

4.6 SELF JOIN

```
-- Employee + their manager (same table)
SELECT e.first_name     AS employee,
       m.first_name     AS manager,
       e.salary
FROM   employees AS e
LEFT JOIN employees AS m ON e.manager_id = m.employee_id;

-- Find employees earning more than their manager
SELECT e.first_name AS employee, e.salary AS emp_salary,
       m.first_name AS manager,  m.salary AS mgr_salary
FROM   employees AS e
JOIN   employees AS m ON e.manager_id = m.employee_id
WHERE  e.salary > m.salary;
```

4.7 Multi-Table JOIN

```
-- Orders → Customers → Products → Categories (4-table join)
SELECT
    o.order_id,
    c.customer_name,
    p.product_name,
    cat.category_name,
    oi.quantity,
    oi.unit_price,
    oi.quantity * oi.unit_price  AS line_total
FROM orders          AS o
JOIN customers       AS c   ON o.customer_id   = c.customer_id
JOIN order_items     AS oi  ON o.order_id       = oi.order_id
JOIN products        AS p   ON oi.product_id    = p.product_id
JOIN categories      AS cat ON p.category_id    = cat.category_id
WHERE o.order_date >= '2024-01-01'
ORDER BY o.order_id;
```

4-table join typical in analytics

5. Subqueries & CTEs

Inline views, scalar subqueries, correlated subqueries, EXISTS, WITH (CTE), recursive CTEs

5.1 Scalar Subquery (returns single value)

```
-- Employees earning above company average
SELECT first_name, salary,
       (SELECT AVG(salary) FROM employees) AS company_avg
FROM   employees
WHERE  salary > (SELECT AVG(salary) FROM employees);
```

5.2 Subquery in FROM (Derived Table / Inline View)

```
-- Department salary summary used as a derived table
SELECT d.department_name, s.avg_salary, s.emp_count
FROM   departments AS d
JOIN   (
    SELECT department_id,
           AVG(salary)  AS avg_salary,
           COUNT(*)     AS emp_count
    FROM   employees
    GROUP  BY department_id
) AS s ON d.department_id = s.department_id
ORDER  BY s.avg_salary DESC;
```

5.3 Correlated Subquery

```
-- Employees earning above their OWN department average
SELECT first_name, department_id, salary
FROM   employees AS e
WHERE  salary > (
    SELECT AVG(salary)
    FROM   employees AS e2
    WHERE  e2.department_id = e.department_id  -- reference outer query
);

-- NOTE: correlated subqueries run once per outer row – can be slow.
-- Rewrite with window functions for better performance.
```

5.4 EXISTS / NOT EXISTS

```
-- Customers who have placed at least one order
SELECT c.customer_id, c.customer_name
FROM   customers AS c
WHERE  EXISTS (
    SELECT 1
    FROM   orders AS o
    WHERE  o.customer_id = c.customer_id
);

-- Customers who have NEVER placed an order
SELECT c.customer_id, c.customer_name
FROM   customers AS c
WHERE  NOT EXISTS (
```

```
    SELECT 1
    FROM   orders AS o
    WHERE  o.customer_id = c.customer_id
);
```

5.5 CTE – Common Table Expression (WITH Clause)

```
-- Single CTE
WITH dept_avg AS (
    SELECT department_id, AVG(salary) AS avg_sal
    FROM   employees
    GROUP  BY department_id
)
SELECT e.first_name, e.salary, d.avg_sal
FROM   employees AS e
JOIN   dept_avg  AS d ON e.department_id = d.department_id
WHERE  e.salary > d.avg_sal;

-- Multiple CTEs (chained)
WITH
revenue AS (
    SELECT order_date, SUM(amount) AS daily_rev
    FROM   orders GROUP BY order_date
),
monthly AS (
    SELECT DATE_TRUNC('month', order_date) AS month,
           SUM(daily_rev) AS monthly_rev
    FROM   revenue
    GROUP  BY 1
)
SELECT month,
       monthly_rev,
       LAG(monthly_rev) OVER (ORDER BY month) AS prev_month,
       monthly_rev - LAG(monthly_rev) OVER (ORDER BY month) AS mom_change
FROM   monthly;
```

CTEs make complex queries readable and reusable

5.6 Recursive CTE

```
-- Traverse org chart hierarchy (employee → manager → ... → CEO)
WITH RECURSIVE org_chart AS (
    -- Anchor: start with the top-level CEO (no manager)
    SELECT employee_id, first_name, manager_id, 1 AS level
    FROM   employees
    WHERE  manager_id IS NULL

    UNION ALL

    -- Recursive: join each employee to their manager's row
    SELECT e.employee_id, e.first_name, e.manager_id, oc.level + 1
    FROM   employees    AS e
    JOIN   org_chart    AS oc ON e.manager_id = oc.employee_id
)
SELECT level, first_name, employee_id
FROM   org_chart
```

```
ORDER  BY level, first_name;

-- Generate number series 1–100 (PostgreSQL)
WITH RECURSIVE nums AS (
    SELECT 1 AS n
    UNION ALL
    SELECT n + 1 FROM nums WHERE n < 100
)
SELECT n FROM nums;
```

6. Window Functions

OVER, PARTITION BY, ORDER BY, frame clauses, ranking, lead/lag, running totals

Window functions perform calculations across a set of rows related to the current row — without collapsing rows like GROUP BY does. They are the most powerful tool in analytical SQL.

6.1 Anatomy of a Window Function

```
function_name(expression)
    OVER (
        [PARTITION BY col1, col2]   -- divide into groups
        [ORDER BY col3 DESC]        -- sort within group
        [ROWS|RANGE BETWEEN ...]    -- optional frame
    )
```

Window function syntax template

6.2 Ranking Functions

```
SELECT
    employee_id,
    department_id,
    salary,
    -- Rank within each department by salary descending
    ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY salary DESC) AS row_num,
    RANK()       OVER (PARTITION BY department_id ORDER BY salary DESC) AS rnk,
    DENSE_RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) AS dense_rnk,
    NTILE(4)     OVER (PARTITION BY department_id ORDER BY salary DESC) AS quartile,
    PERCENT_RANK() OVER (PARTITION BY department_id ORDER BY salary)    AS pct_rank
FROM employees;

-- ROW_NUMBER : 1,2,3,4    (no ties – always unique)
-- RANK       : 1,1,3,4    (gaps after ties)
-- DENSE_RANK : 1,1,2,3    (no gaps)
-- NTILE(4)   : splits into 4 buckets (quartiles)

-- Get top-2 earners per department
SELECT * FROM (
    SELECT *, DENSE_RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) AS dr
    FROM employees
) t WHERE dr <= 2;
```

6.3 LAG & LEAD – Access Adjacent Rows

```
SELECT
    order_date,
    revenue,
    LAG(revenue)              OVER (ORDER BY order_date) AS prev_day_rev,
    LEAD(revenue)             OVER (ORDER BY order_date) AS next_day_rev,
    LAG(revenue, 7)           OVER (ORDER BY order_date) AS same_day_last_week,
    revenue - LAG(revenue)    OVER (ORDER BY order_date) AS day_over_day_change,
    ROUND(
        100.0*(revenue - LAG(revenue) OVER (ORDER BY order_date))
        / NULLIF(LAG(revenue) OVER (ORDER BY order_date), 0)
```

```
    , 2)                                                AS pct_change
FROM daily_sales;
```

6.4 Running Totals & Moving Averages

```
SELECT
    order_date,
    revenue,
    SUM(revenue)  OVER (ORDER BY order_date
                        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)  AS running_total,
    AVG(revenue)  OVER (ORDER BY order_date
                        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)          AS ma_7day,
    SUM(revenue)  OVER (PARTITION BY DATE_TRUNC('month', order_date)
                        ORDER BY order_date
                        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)  AS month_running_total
FROM daily_sales;

-- Frame clause options:
-- ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW   (running total)
-- ROWS BETWEEN 6 PRECEDING AND CURRENT ROW           (7-day window)
-- ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING (whole partition)
```

6.5 FIRST_VALUE, LAST_VALUE, NTH_VALUE

```
SELECT
    employee_id, department_id, salary,
    FIRST_VALUE(salary) OVER (PARTITION BY department_id
                              ORDER BY salary DESC)                          AS dept_max_salary,
    LAST_VALUE(salary)  OVER (PARTITION BY department_id
                              ORDER BY salary DESC
                              ROWS BETWEEN UNBOUNDED PRECEDING
                                       AND UNBOUNDED FOLLOWING)              AS dept_min_salary,
    NTH_VALUE(salary, 2) OVER (PARTITION BY department_id
                               ORDER BY salary DESC)                         AS second_highest
FROM employees;
```

6.6 Aggregate Window Functions

```
-- All aggregates work as window functions
SELECT
    employee_id, department_id, salary,
    SUM(salary)   OVER (PARTITION BY department_id)          AS dept_total,
    AVG(salary)   OVER (PARTITION BY department_id)          AS dept_avg,
    COUNT(*)      OVER (PARTITION BY department_id)          AS dept_headcount,
    salary / SUM(salary) OVER (PARTITION BY department_id)   AS salary_share,
    -- No PARTITION BY = across entire result set
    salary / SUM(salary) OVER ()                             AS company_share
FROM employees;
```

7. Date & Time Functions

Extraction, arithmetic, formatting, date truncation, calendar logic

7.1 Getting Current Date / Time

```
-- Current date and time
SELECT CURRENT_DATE;                    -- standard SQL
SELECT CURRENT_TIMESTAMP;               -- standard SQL
SELECT NOW();                           -- MySQL, PostgreSQL
SELECT GETDATE();                       -- SQL Server
SELECT SYSDATE FROM DUAL;              -- Oracle
SELECT CURRENT_DATE();                 -- BigQuery
```

7.2 Extracting Date Parts

```
-- Standard EXTRACT (works in most databases)
SELECT
    EXTRACT(YEAR  FROM order_date) AS yr,
    EXTRACT(MONTH FROM order_date) AS mo,
    EXTRACT(DAY   FROM order_date) AS dy,
    EXTRACT(HOUR  FROM order_timestamp) AS hr,
    EXTRACT(DOW   FROM order_date) AS day_of_week  -- 0=Sun in PostgreSQL

-- SQL Server
SELECT
    YEAR(order_date),
    MONTH(order_date),
    DAY(order_date),
    DATEPART(WEEKDAY, order_date),
    DATEPART(QUARTER, order_date),
    DATEPART(WEEK, order_date)

-- Date truncation (floor to period start)
SELECT DATE_TRUNC('month',  order_date)   -- PostgreSQL, BigQuery
SELECT DATE_TRUNC('quarter',order_date)
SELECT DATETRUNC(month,     order_date)   -- SQL Server 2022+
SELECT TRUNC(order_date, 'MM')            -- Oracle
```

7.3 Date Arithmetic

```
-- Days between dates
SELECT DATEDIFF('2024-12-31','2024-01-01');    -- MySQL → 365
SELECT DATEDIFF(day, '2024-01-01','2024-12-31'); -- SQL Server → 365
SELECT '2024-12-31'::date - '2024-01-01'::date;  -- PostgreSQL → 365

-- Add / subtract intervals
SELECT order_date + INTERVAL 30 DAY          -- MySQL, PostgreSQL
SELECT DATEADD(day, 30, order_date)          -- SQL Server
SELECT order_date + 30                       -- Oracle (adds days)
SELECT DATE_ADD(order_date, INTERVAL 1 MONTH) -- MySQL

-- Age / tenure calculation
SELECT DATEDIFF(CURRENT_DATE, hire_date) / 365.25 AS years_employed  -- MySQL
```

```
SELECT DATE_DIFF(CURRENT_DATE, hire_date, YEAR)                    -- BigQuery
SELECT DATEDIFF(year, hire_date, GETDATE())                        -- SQL Server
```

7.4 Formatting Dates

```
-- SQL Server
SELECT FORMAT(order_date, 'yyyy-MM-dd')   -- 2024-06-15
SELECT FORMAT(order_date, 'MMMM yyyy')    -- June 2024
SELECT CONVERT(VARCHAR, order_date, 103)  -- dd/MM/yyyy

-- MySQL
SELECT DATE_FORMAT(order_date, '%Y-%m-%d')     -- 2024-06-15
SELECT DATE_FORMAT(order_date, '%W, %M %d %Y') -- Saturday, June 15 2024

-- PostgreSQL
SELECT TO_CHAR(order_date, 'YYYY-MM-DD')       -- 2024-06-15
SELECT TO_CHAR(order_date, 'Day, DD Mon YYYY') -- Saturday, 15 Jun 2024

-- Parse string to date
SELECT STR_TO_DATE('15/06/2024', '%d/%m/%Y')   -- MySQL
SELECT TO_DATE('15/06/2024', 'DD/MM/YYYY')     -- Oracle, PostgreSQL
SELECT CAST('2024-06-15' AS DATE)              -- All databases
```

7.5 Useful Date Analytics Patterns

```
-- First and last day of current month
SELECT DATE_TRUNC('month', CURRENT_DATE)                                    AS month_start,
       DATE_TRUNC('month', CURRENT_DATE) + INTERVAL '1 month' - INTERVAL '1 day' AS month_end;

-- Same period last year
WHERE order_date BETWEEN
    DATE_ADD(CURRENT_DATE, INTERVAL -1 YEAR - DAY(CURRENT_DATE)+1 DAY)
    AND DATE_ADD(CURRENT_DATE, INTERVAL -1 YEAR)

-- Last 30 / 90 / 365 days
WHERE order_date >= CURRENT_DATE - INTERVAL '30 days'

-- Week number and ISO week
SELECT EXTRACT(WEEK FROM order_date)     AS week_num,
       EXTRACT(ISODOW FROM order_date)   AS iso_day  -- 1=Mon, 7=Sun (PostgreSQL)
```

8. String Functions

Concatenation, slicing, searching, trimming, replacing, splitting, regex

8.1 Common String Functions Reference

Function	Purpose	Example
LEN / LENGTH(s)	String length	LEN('Hello') → 5
UPPER(s)	Uppercase	UPPER('sql') → SQL
LOWER(s)	Lowercase	LOWER('SQL') → sql
TRIM(s)	Remove leading/trailing spaces	TRIM(' hi ') → hi
LTRIM / RTRIM	Remove one side only	LTRIM(' hi') → hi
LEFT(s,n)	First n chars	LEFT('Hello',3) → Hel
RIGHT(s,n)	Last n chars	RIGHT('Hello',3) → llo
SUBSTRING(s,pos,len)	Extract substring	SUBSTRING('Hello',2,3) → ell
CHARINDEX / INSTR	Position of substring	CHARINDEX('l','Hello') → 3
REPLACE(s,old,new)	Replace text	REPLACE('foo','o','0') → f00
CONCAT(s1,s2,...)	Join strings	CONCAT('Hi',' ','SQL') → Hi SQL
CONCAT_WS(sep,...)	Join with separator	CONCAT_WS(',','A','B') → A,B
REVERSE(s)	Reverse string	REVERSE('abc') → cba
REPEAT/REPLICATE(s,n)	Repeat string	REPLICATE('ab',3) → ababab
FORMAT/PRINTF	Format numbers	FORMAT(1234567,'N0') → 1,234,567
CAST / CONVERT	Type conversion	CAST(3.14 AS INT) → 3
COALESCE(a,b,c)	First non-NULL	COALESCE(NULL,NULL,'hi') → hi
NULLIF(a,b)	Return NULL if a=b	NULLIF(0,0) → NULL

8.2 String Functions in Practice

```
-- Clean and standardise names
SELECT
    TRIM(UPPER(first_name))                    AS clean_first,
    TRIM(LOWER(email))                         AS clean_email,
    CONCAT(first_name, ' ', last_name)         AS full_name,
    LEFT(phone, 3)                             AS area_code,
    REPLACE(REPLACE(phone,'(',''),')','')      AS clean_phone,
    SUBSTRING(postal_code, 1, 5)               AS zip5

FROM customers;
```

```
-- Extract domain from email
SELECT
    email,
    SUBSTRING(email, CHARINDEX('@', email)+1, LEN(email)) AS domain  -- SQL Server
    -- SUBSTRING(email FROM POSITION('@' IN email)+1)                 -- PostgreSQL
FROM customers;

-- Split full name into first/last (SQL Server)
SELECT
    full_name,
    LEFT(full_name, CHARINDEX(' ', full_name)-1)           AS first_name,
    RIGHT(full_name, LEN(full_name)-CHARINDEX(' ',full_name)) AS last_name
FROM contacts;
```

8.3 Regular Expressions

```
-- PostgreSQL
SELECT * FROM emails WHERE email ~ '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}$';
SELECT REGEXP_REPLACE(phone,'[^0-9]','','g') AS digits_only FROM contacts;
SELECT REGEXP_EXTRACT(text, '[0-9]+')        AS first_number FROM logs;

-- MySQL
SELECT * FROM users WHERE phone REGEXP '^[0-9]{10}$';
SELECT REGEXP_REPLACE(col, '[aeiou]', '*') FROM text_data;

-- BigQuery
SELECT REGEXP_EXTRACT(url, r'utm_source=([^&]+)') AS utm_source FROM events;
SELECT REGEXP_CONTAINS(email, r'@gmail\.com')     AS is_gmail   FROM users;
```

9. Conditional Logic

CASE WHEN, IIF, COALESCE, NULLIF, ISNULL/NVL, NULL handling

9.1 CASE WHEN – If-Then-Else

```
-- Simple CASE (equality check)
SELECT product_name,
       CASE category_id
           WHEN 1 THEN 'Electronics'
           WHEN 2 THEN 'Clothing'
           WHEN 3 THEN 'Food'
           ELSE       'Other'
       END AS category_label
FROM products;

-- Searched CASE (range / complex conditions)
SELECT first_name, salary,
       CASE
           WHEN salary < 30000              THEN 'Entry Level'
           WHEN salary BETWEEN 30000 AND 60000 THEN 'Mid Level'
           WHEN salary BETWEEN 60001 AND 100000 THEN 'Senior'
           WHEN salary > 100000             THEN 'Executive'
           ELSE 'Unknown'
       END AS salary_band
FROM employees;

-- CASE in aggregation (conditional counting)
SELECT
    department_id,
    COUNT(*)                                    AS total_employees,
    COUNT(CASE WHEN salary > 70000 THEN 1 END)   AS high_earners,
    SUM(CASE WHEN gender='F' THEN salary ELSE 0 END) AS female_payroll,
    AVG(CASE WHEN job_id='MGR' THEN salary END)  AS avg_manager_salary
FROM employees
GROUP BY department_id;
```

9.2 NULL Handling Functions

```
-- COALESCE: return first non-NULL value (ANSI standard – use this)
SELECT COALESCE(phone_mobile, phone_home, phone_work, 'No Phone') AS contact
FROM customers;

-- ISNULL (SQL Server)          -- NVL (Oracle)         -- IFNULL (MySQL)
SELECT ISNULL(discount, 0)      -- NVL(discount,0)      -- IFNULL(discount,0)
FROM orders;

-- NULLIF: return NULL when two values are equal (prevents divide-by-zero)
SELECT sales / NULLIF(target, 0) AS attainment_ratio
FROM performance;

-- Handling NULLs in aggregations
-- COUNT(*) counts NULLs; COUNT(col) does NOT
```

```
-- SUM/AVG/MIN/MAX automatically ignore NULLs
SELECT
    COUNT(*)           AS all_rows,
    COUNT(email)       AS rows_with_email,
    COUNT(*) - COUNT(email) AS missing_emails
FROM customers;

-- NULL in comparisons -- always use IS NULL / IS NOT NULL
WHERE email  IS NULL          -- correct
WHERE email = NULL            -- WRONG – always false!
```

9.3 IIF (SQL Server / Access)

```
-- IIF is shorthand for simple CASE WHEN
SELECT name,
       IIF(salary > 60000, 'High', 'Low') AS salary_tier
FROM employees;

-- Equivalent CASE WHEN
SELECT name,
       CASE WHEN salary > 60000 THEN 'High' ELSE 'Low' END AS salary_tier
FROM employees;
```

9.4 PIVOT / Conditional Aggregation

```
-- Pivot months into columns using conditional aggregation
SELECT
    product_id,
    SUM(CASE WHEN month_num=1  THEN revenue ELSE 0 END) AS jan,
    SUM(CASE WHEN month_num=2  THEN revenue ELSE 0 END) AS feb,
    SUM(CASE WHEN month_num=3  THEN revenue ELSE 0 END) AS mar,
    SUM(CASE WHEN month_num=4  THEN revenue ELSE 0 END) AS apr,
    SUM(revenue)                                        AS ytd_total
FROM monthly_sales
GROUP BY product_id;

-- SQL Server PIVOT operator
SELECT *
FROM monthly_sales
PIVOT (
    SUM(revenue) FOR month_num IN ([1],[2],[3],[4],[5],[6],[7],[8],[9],[10],[11],[12])
) AS pvt;
```

10. Advanced Analytics Patterns

Cohort analysis, funnel analysis, retention, YoY/MoM, percentiles, sessionisation

10.1 Year-over-Year & Month-over-Month Comparison

```
WITH monthly AS (
    SELECT
        DATE_TRUNC('month', order_date) AS month,
        SUM(revenue)                    AS revenue
    FROM orders
    GROUP BY 1
)
SELECT
    month,
    revenue,
    LAG(revenue, 1) OVER (ORDER BY month)  AS prev_month_rev,
    LAG(revenue, 12) OVER (ORDER BY month) AS same_month_last_year,
    ROUND(100.0*(revenue - LAG(revenue,1) OVER (ORDER BY month))
          / NULLIF(LAG(revenue,1) OVER (ORDER BY month),0),2) AS mom_pct,
    ROUND(100.0*(revenue - LAG(revenue,12) OVER (ORDER BY month))
          / NULLIF(LAG(revenue,12) OVER (ORDER BY month),0),2) AS yoy_pct
FROM monthly
ORDER BY month;
```

10.2 Cohort Retention Analysis

```
-- Step 1: Assign each user their cohort (first purchase month)
WITH cohorts AS (
    SELECT
        customer_id,
        MIN(DATE_TRUNC('month', order_date)) AS cohort_month
    FROM orders
    GROUP BY customer_id
),
-- Step 2: Join back to find activity in subsequent months
activity AS (
    SELECT
        c.customer_id,
        c.cohort_month,
        DATE_TRUNC('month', o.order_date) AS order_month,
        DATEDIFF(month, c.cohort_month, DATE_TRUNC('month', o.order_date)) AS period
    FROM cohorts c
    JOIN orders o ON c.customer_id = o.customer_id
)
-- Step 3: Aggregate retention
SELECT
    cohort_month,
    period,
    COUNT(DISTINCT customer_id)                                    AS active_users,
    COUNT(DISTINCT customer_id) * 100.0
        / FIRST_VALUE(COUNT(DISTINCT customer_id))
          OVER (PARTITION BY cohort_month ORDER BY period)         AS retention_pct
```

```
FROM activity
GROUP BY cohort_month, period
ORDER BY cohort_month, period;
```

Classic cohort retention table

10.3 Funnel Analysis

```
-- Conversion funnel: Visit → SignUp → Purchase → Repeat
WITH funnel AS (
    SELECT
        user_id,
        MAX(CASE WHEN event='page_visit'   THEN 1 ELSE 0 END) AS visited,
        MAX(CASE WHEN event='signup'       THEN 1 ELSE 0 END) AS signed_up,
        MAX(CASE WHEN event='purchase'     THEN 1 ELSE 0 END) AS purchased,
        MAX(CASE WHEN event='2nd_purchase' THEN 1 ELSE 0 END) AS repeated
    FROM events
    GROUP BY user_id
)
SELECT
    SUM(visited)   AS step1_visitors,
    SUM(signed_up) AS step2_signups,
    SUM(purchased) AS step3_purchases,
    SUM(repeated)  AS step4_repeats,
    ROUND(100.0*SUM(signed_up) /NULLIF(SUM(visited)  ,0),1) AS visit_to_signup_pct,
    ROUND(100.0*SUM(purchased) /NULLIF(SUM(signed_up),0),1) AS signup_to_purchase_pct,
    ROUND(100.0*SUM(repeated)  /NULLIF(SUM(purchased),0),1) AS repeat_rate_pct
FROM funnel;
```

10.4 Percentile & Distribution Analysis

```
-- Percentiles (PostgreSQL, BigQuery)
SELECT
    PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY salary) AS p25,
    PERCENTILE_CONT(0.50) WITHIN GROUP (ORDER BY salary) AS median,
    PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY salary) AS p75,
    PERCENTILE_CONT(0.90) WITHIN GROUP (ORDER BY salary) AS p90,
    PERCENTILE_CONT(0.95) WITHIN GROUP (ORDER BY salary) AS p95
FROM employees;

-- Using window function (works in more dialects)
SELECT
    salary,
    NTILE(4)  OVER (ORDER BY salary) AS quartile,
    NTILE(10) OVER (ORDER BY salary) AS decile,
    NTILE(100)OVER (ORDER BY salary) AS percentile
FROM employees;

-- SQL Server approximation
SELECT
    DISTINCT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary)
             OVER (PARTITION BY department_id) AS dept_median_salary
FROM employees;
```

10.5 Sessionisation (Gap & Island)

```
-- Assign session IDs: new session if gap > 30 min
WITH gaps AS (
    SELECT user_id, event_time,
           LAG(event_time) OVER (PARTITION BY user_id ORDER BY event_time) AS prev_time,
           CASE WHEN DATEDIFF(minute,
                     LAG(event_time) OVER (PARTITION BY user_id ORDER BY event_time),
                     event_time) > 30
                     OR LAG(event_time) OVER (PARTITION BY user_id ORDER BY event_time) IS NULL
                THEN 1 ELSE 0 END AS is_new_session
    FROM events
),
sessions AS (
    SELECT *,
           SUM(is_new_session) OVER (PARTITION BY user_id ORDER BY event_time) AS session_id
    FROM gaps
)
SELECT user_id, session_id, MIN(event_time) AS session_start,
       MAX(event_time) AS session_end,
       COUNT(*) AS events_in_session,
       DATEDIFF(minute, MIN(event_time), MAX(event_time)) AS duration_min
FROM sessions
GROUP BY user_id, session_id;
```

11. Data Cleaning in SQL

NULL handling, deduplication, type casting, standardisation, outlier detection

11.1 Detecting Data Quality Issues

```
-- Null audit across all columns
SELECT
    COUNT(*)                              AS total_rows,
    COUNT(*) - COUNT(customer_id)         AS null_customer_id,
    COUNT(*) - COUNT(email)               AS null_email,
    COUNT(*) - COUNT(phone)               AS null_phone,
    COUNT(*) - COUNT(signup_date)         AS null_signup_date,
    ROUND(100.0*(COUNT(*)-COUNT(email))/COUNT(*),1) AS pct_missing_email
FROM customers;

-- Duplicate detection
SELECT email, COUNT(*) AS occurrences
FROM customers
GROUP BY email
HAVING COUNT(*) > 1
ORDER BY occurrences DESC;
```

11.2 Deduplication

```
-- Keep only the most recent record per customer (CTE + ROW_NUMBER)
WITH ranked AS (
    SELECT *,
           ROW_NUMBER() OVER (PARTITION BY customer_id
                              ORDER BY updated_at DESC) AS rn
    FROM customers
)
SELECT * FROM ranked WHERE rn = 1;

-- Delete duplicates keeping lowest ID (SQL Server / PostgreSQL)
DELETE FROM customers
WHERE customer_id NOT IN (
    SELECT MIN(customer_id)
    FROM customers
    GROUP BY email
);
```

11.3 Type Casting & Conversion

```
-- CAST (ANSI standard)
SELECT CAST('2024-01-15' AS DATE)       AS date_val,
       CAST('3.14'       AS DECIMAL(5,2)) AS num_val,
       CAST(salary       AS VARCHAR(20))  AS salary_str;

-- TRY_CAST / TRY_CONVERT (SQL Server) – returns NULL instead of error
SELECT TRY_CAST(messy_col AS INT) AS safe_int FROM raw_data;

-- PostgreSQL casting shorthand
SELECT '3.14'::DECIMAL(5,2), order_date::VARCHAR;
```

```
-- Safe numeric conversion check
SELECT col,
       CASE WHEN col NOT LIKE '%[^0-9]%' AND col != ''
            THEN CAST(col AS INT) ELSE NULL END AS safe_int
FROM raw_data;
```

11.4 Standardising & Cleaning Values

```
-- Standardise free-text fields
SELECT
    customer_id,
    TRIM(UPPER(REPLACE(REPLACE(gender,'male','M'),'female','F'))) AS gender_clean,
    CASE
        WHEN LOWER(country) IN ('us','usa','united states','america') THEN 'United States'
        WHEN LOWER(country) IN ('uk','gb','united kingdom','britain') THEN 'United Kingdom'
        ELSE TRIM(INITCAP(country))    -- PostgreSQL: capitalize first letter
    END AS country_clean,
    REGEXP_REPLACE(phone,'[^0-9]','') AS phone_digits   -- digits only
FROM customers;

-- Flag outliers (IQR method)
WITH stats AS (
    SELECT
        PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY salary) AS q1,
        PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY salary) AS q3
    FROM employees
)
SELECT e.*, s.q1, s.q3,
       CASE WHEN salary < s.q1 - 1.5*(s.q3-s.q1)
              OR salary > s.q3 + 1.5*(s.q3-s.q1)
            THEN 'Outlier' ELSE 'Normal' END AS outlier_flag
FROM employees e CROSS JOIN stats s;
```

12. Performance & Best Practices

Indexing, query optimisation, anti-patterns, UNION, EXPLAIN, tips

12.1 UNION & UNION ALL

```
-- UNION removes duplicates (slower)
SELECT customer_id FROM current_customers
UNION
SELECT customer_id FROM archived_customers;

-- UNION ALL keeps duplicates (faster – use when you know data is distinct)
SELECT 'current' AS source, customer_id, name FROM current_customers
UNION ALL
SELECT 'archived',          customer_id, name FROM archived_customers;

-- Stack multiple result sets
SELECT 'Q1' AS quarter, SUM(revenue) FROM sales WHERE quarter=1
UNION ALL
SELECT 'Q2',            SUM(revenue) FROM sales WHERE quarter=2
UNION ALL
SELECT 'Q3',            SUM(revenue) FROM sales WHERE quarter=3
UNION ALL
SELECT 'Q4',            SUM(revenue) FROM sales WHERE quarter=4;
```

12.2 INTERSECT & EXCEPT (MINUS)

```
-- Customers who exist in BOTH tables
SELECT customer_id FROM online_orders
INTERSECT
SELECT customer_id FROM store_orders;

-- Customers in online but NOT in store (EXCEPT = MINUS in Oracle)
SELECT customer_id FROM online_orders
EXCEPT          -- use MINUS in Oracle
SELECT customer_id FROM store_orders;
```

12.3 Performance Anti-Patterns to Avoid

Anti-Pattern	Problem	Better Approach
SELECT *	Pulls all columns; breaks cached plans	List only needed columns
Function on indexed col in WHERE	Disables index seek	Rewrite: col BETWEEN x AND y
Correlated subquery in SELECT	Runs once per row	Use JOIN or window function
NOT IN with NULLs	Returns 0 rows if list has NULL	Use NOT EXISTS instead
Implicit type conversion	Index not used	CAST explicitly or match types
OR on different columns	Can't use composite index	Use UNION ALL instead
DISTINCT as a fix	Masks JOIN problems	Find and fix root join issue

ORDER BY in subquery	No guarantee + wastes resources	Sort in outermost query only
Wildcard LIKE '%term%'	Full table scan (no index)	Use full-text search instead

12.4 EXPLAIN / Query Plan

```
-- PostgreSQL / MySQL
EXPLAIN SELECT * FROM orders WHERE customer_id = 1001;
EXPLAIN ANALYZE SELECT * FROM orders WHERE customer_id = 1001;  -- actually runs it

-- SQL Server
SET STATISTICS IO ON;
SET STATISTICS TIME ON;
SELECT * FROM orders WHERE customer_id = 1001;

-- Look for: Table Scan (bad) vs Index Seek/Scan (good)
-- High cost operations: Hash Match, Sort, Nested Loops on large tables
```

12.5 Useful Analytical SQL Cheatsheet

Pattern	SQL Technique	Use Case
Top N per group	ROW_NUMBER + filter	Analytics, rankings
Running total	SUM OVER (ORDER BY)	Finance, inventory
Moving average	AVG OVER (ROWS 6 PRECEDING)	Time series smoothing
Period-over-period	LAG(col, 12) OVER	MoM, YoY reports
De-duplicate	ROW_NUMBER PARTITION BY key	Data cleaning
Fill NULLs (forward fill)	LAST_VALUE IGNORE NULLS	Time series gaps
Find gaps	LAG/LEAD + DATEDIFF	Session analysis
Pivot rows to columns	Conditional SUM(CASE)	Reporting
Unpivot columns to rows	UNION ALL or UNPIVOT	Normalisation
Hierarchical traverse	Recursive CTE	Org charts, BOM
Median	PERCENTILE_CONT(0.5)	Statistics
Cohort retention	DATEDIFF to cohort month	User analytics

12.6 SQL Dialect Quick Reference

Feature	MySQL / BigQuery	SQL Server	Oracle
Limit rows	LIMIT n	TOP n / FETCH FIRST n	ROWNUM / FETCH FIRST
String concat	CONCAT() or ||	+ operator	|| or CONCAT()
Current date	CURRENT_DATE / NOW()	GETDATE()	SYSDATE
If null	IFNULL / COALESCE	ISNULL / COALESCE	NVL / COALESCE

Auto-increment	AUTO_INCREMENT	IDENTITY(1,1)	SEQUENCE / GENERATED
Regex match	REGEXP / RLIKE	LIKE (basic only)	REGEXP_LIKE
Date diff	DATEDIFF(unit,d1,d2)	DATEDIFF(d1,d2)	MONTHS_BETWEEN
Pivot	Manual CASE SUM	PIVOT operator	Manual CASE SUM
Recursive CTE	WITH RECURSIVE	WITH (no keyword)	WITH (no keyword)

Tip: Write readable SQL: use consistent capitalisation for keywords, one clause per line, meaningful aliases, and CTEs to break up complex logic. Future-you (and your teammates) will thank you.